VQE should batch per default

https://github.com/Qiskit/qiskit/issues/8926

ROW 55

VQE should batch per default #8926

New issue

Closed #9038

Cryoris opened on Oct 17, 2022

What should we add?

The VQE is most commonly used with SPSA or gradient descent, which require 2 or (up to) 2 * `num_parameters` circuit evaluations per iterations, respectively. Since the new execution model is shifting from using the runtime VQE program to the primitive-based VQE in Terra, we should also enable batching as many circuits per default as possible. Otherwise the standard usage of the VQE will be much slower than necessary (~2 or 2 * `num_parameters` to slow 😅).

This is something the runtime VQE also did, maybe we can use the same defaults?

Cryoris added type: feature request on Oct 17, 2022

Cryoris on Oct 17, 2022 Contributor Author

Actually, maybe we can consider this as performance bug and put it in the 0.22.1 release for November 🤔

Cryoris added bug and removed type: feature request on Oct 17, 2022

Cryoris added this to the 0.22.1 milestone on Oct 17, 2022

mtreinish on Oct 17, 2022 Member

As long as we don't add any new flags or change the api I think it's fine to backport and include this in 0.22.1 as a performance fix.

👍 1

woodsp-ibm added mod: algorithms on Oct 25, 2022

Cryoris mentioned this on Oct 31, 2022

Fix default batching in variational algorithms #9038

mergify closed this as completed in #9038 on Nov 1, 2022

Assignees

No one assigned

Labels

bug mod: algorithms

Type

No type

Projects

No projects

Milestone

0.22.1

Closed on Nov 2, 2022, 100% complete

Relationships

None yet

Development

Fix default batching in variational algorithms

Qiskit/qiskit

Notifications Customize

Subscribe

You're not receiving notifications from this thread.

Participants

Fix default batching in variational algorithms #9038

Code

Merged mergify merged 6 commits into `Qiskit:main` from `Cryoris:fix-vqe-batching` on Nov 1, 2022

Conversation 7 · Commits 6 · Checks 0 · Files changed 8 · +103 −4

Cryoris commented on Oct 31, 2022 — Contributor

Summary

Fixes #8926.

Details and comments

As we motivate users to use the runtime primitives over runtime programs, it is important that the code remains as efficient as possible. With the variational algorithms released in 0.22.0 however, there's a significant slowdown if hardware is used (also with Sessions), as we didn't include batching of energy evaluations. This PR fixes that performance issue by adding default batching in case the user didn't specify any, which is the same behavior users previously had with the runtime programs.

For the 0.23 release we're planning to update how batching works and then these hardcoded limits can be removed, but we probably don't want to wait 3 months to fix this slowdown of a factor of ~2 😬

I'm not sure if it's ok to change the default of the internal `_max_evals_grouped` (@mtreinish @jakelishman what do you think?) but I couldn't find another way to check if the user set the `_max_evals_grouped` manually. If that change blocks inlcuding this as a bugfix, we could also *not* fix this behavior and add a very big textbox in the docs that says users really have to enable batching themselves manually. Until they discover that note, however, they likely already complained about slow runtimes... 😬

`Fix default batching in variational algorithms` Unverified 757e794

Cryoris requested review from **a team**, **manoelmarques** and **woodsp-ibm** as code owners 3 years ago

qiskit-bot commented on Oct 31, 2022 — Collaborator

Thank you for opening a new pull request.

Before your PR can be merged it will first need to pass continuous integration tests and be reviewed. Sometimes the review process can be slow, so please be patient.

While you're waiting, please feel free to review other open PRs. While only a subset of people are authorized to approve pull requests for merging, everyone is encouraged to review open pull requests. Doing reviews helps reduce the burden on the core team and helps make the project's code better for everyone.

One or more of the the following people are requested to review this:

- @Qiskit/terra-core
- @manoelmarques
- @woodsp-ibm

Reviewers
- woodsp-ibm ✓
- mtreinish ✓
- manoelmarques ●

Assignees
No one assigned

Labels
Changelog: Bugfix · mod: algorithms · stable backport potential

Projects
None yet

Milestone
0.22.1

Development
Successfully merging this pull request may close these issues.
VQE should batch per default

Notifications Customize
Subscribe
You're not receiving notifications from this thread.

5 participants

woodsp-ibm commented on Oct 31, 2022 Member

When we first put such function in Aqua it was a simple boolean to turn on batching or not - the optimizer could send as many as it wanted when enabled. People wanted better performance and we were looking to leverage Aer parallel capability. What transpired was that the performance consideration was in regard of running bigger systems - larger operator and more pauli terms/groups. What happened was it would run out of memory with the circuits very easily. Hence we dropped to max_evals_grouped as a more controlled setting and defaulted it off - here the number of points (evals) could be defined, with it defaulting to 1 which is normal non-batched behavior.

Could something be more specifically done around SPSA that is being used for real hardware - detect that and set the batch level to 2 for your default case. I guess that does not cover the calibration where we can batch all those points at once. If that's needed then enough to cater to that too. Hopefully whatever we do then with some remote device, whether via the runtime or via say these backend wrappers we do not hit memory issues. I guess if that would happen then setting the max grouped evals explicitly on SPSA to some smaller number for a user would be a solution should this be a problem. This way hopefully it performs by default well, compared to the original runtime VQE etc, and there is some fallback, which the code in this PR already caters too in terms of an explicit setting. Since the runtime devices are all real/qasm sim (not statevector) maybe limiting this to SPSA might be ok - though I believe you had a concern over the gradient descent too but maybe that can be included too so its more selective about the optimizer as to when its enabled and maybe the values set can be chosen accordingly.

Cryoris commented on Nov 1, 2022 Contributor Author

After talking with @jyu00 and running a test with 1000 circuits on the primitives (which failed) maybe reducing the number of grouped evals specific to SPSA would be the best approach for now. Until the primitives handle chopping the jobs into sizes that the memory on the devices can handle, we cannot just "turn on" the batching as we still have the same problem @woodsp-ibm describes above. I'll adjust it 👍🏻

Cryoris added 2 commits 3 years ago

reduce batching to only SPSA	Unverified	44f86a6
fix tests	Unverified	e0e6837

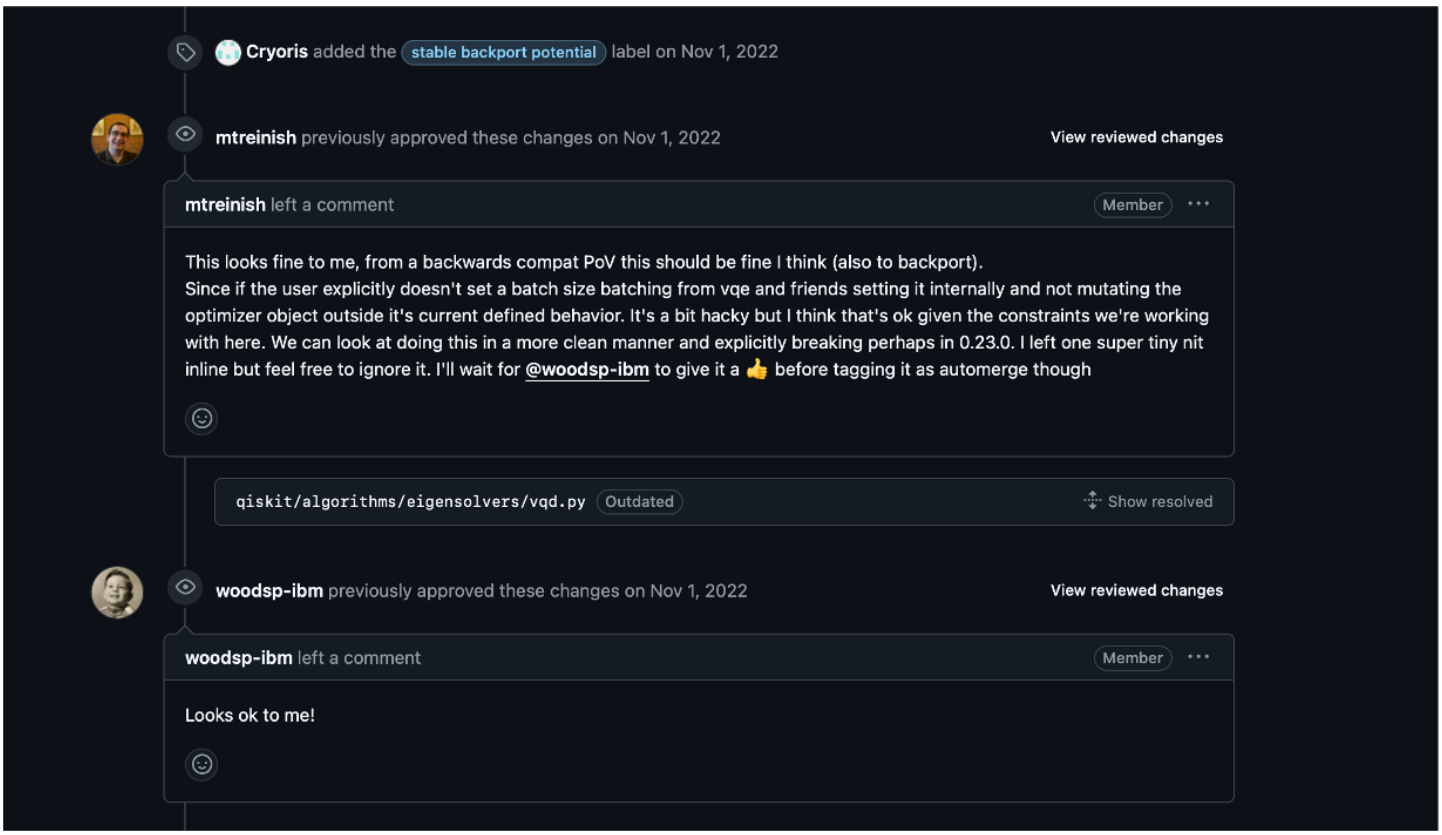
coveralls commented on Nov 1, 2022 • edited

Pull Request Test Coverage Report for Build 3371383702

- **24** of **26** **(92.31%)** changed or added relevant lines in **6** files are covered.
- No unchanged relevant lines lost coverage.
- Overall coverage increased **(+0.009%)** to **84.499%**

Changes Missing Coverage	Covered Lines	Changed/Added Lines	%
qiskit/algorithms/eigensolvers/vqd.py	3	4	75.0%
qiskit/algorithms/minimum_eigensolvers/sampling_vqe.py	3	4	75.0%

Totals	coverage 84%
Change from base Build 3371383153:	0.009%
Covered Lines:	62464
Relevant Lines:	73923


Cryoris added the stable backport potential label on Nov 1, 2022
mtreinish previously approved these changes on Nov 1, 2022
View reviewed changes
mtreinish left a comment
Member
This looks fine to me, from a backwards compat PoV this should be fine I think (also to backport).
Since if the user explicitly doesn't set a batch size batching from vqe and friends setting it internally and not mutating the optimizer object outside it's current defined behavior. It's a bit hacky but I think that's ok given the constraints we're working with here. We can look at doing this in a more clean manner and explicitly breaking perhaps in 0.23.0. I left one super tiny nit inline but feel free to ignore it. I'll wait for @woodsp-ibm to give it a 👍 before tagging it as automerge though
qiskit/algorithms/eigensolvers/vqd.py
Outdated
Show resolved
woodsp-ibm previously approved these changes on Nov 1, 2022
View reviewed changes
woodsp-ibm left a comment
Member
Looks ok to me!